

OPERATING SYSTEMS

(UNIT - IV)

FILE CONCEPTS

The file system is the most visible aspect of an OS. It provides the mechanism for online storage of and access to both data and programs of OS and all the users of the computer system. The file system consists of two distinct parts: a collection of files – each storing related data and a directory structure which organizes and provides information about all the files in the system.

FILE CONCEPTS:

Computers can store information on various storage media such as magnetic disks, magnetic tapes and optical disks. OS provides a uniform logical view of information storage. OS abstracts from the physical properties of its storage devices to define a logical storage unit called a **file**. Files are mapped by OS onto physical devices. These storage devices are non volatile so the contents are persistent through power failures and system reboots.

A file is a named collection of related information that is recorded on secondary storage. A file is the smallest allotment of logical secondary storage; that is data cannot be written to secondary storage unless they are within a file. Files represent programs and data. Data files may be numeric, alphabetic, alphanumeric or binary. Files may be free form such as text files or may be formatted rigidly. A file is a sequence of bits, bytes, lines or records.

Information in a file is defined by its creator. Many different types of information may be stored in a file – source programs, object programs, executable programs, numeric data, text etc. A file has a certain defined structure which depends on its type.

-  Text file –sequence of characters organized into lines
-  Source file – sequence of sub routines and functions each of which is further organized as declarations followed by executable statements.
-  Object file – sequence of bytes organized into blocks understandable by the system's linker
-  Executable file – series of code sections that the loader can bring into memory and execute.

File Attributes

File attributes stores the information about the files. A file is referred to by its name. A name is usually a string of characters. When a file is named, it becomes independent of the process, the user and even the system that created it.

A file's attributes vary from one OS to another but consist of these –

Name: symbolic file name is the only information kept in human readable form.

Identifier: number which identifies the file within the file system; it is the non human readable name for the file.

Type: information is needed for systems that support different types of files.

Location: this information is a pointer to a device and to the location of the file on that device.

Size: the current size of the file

Protection: Access control information determines who can do reading, writing, executing etc.

Time, date and user identification: This information may be kept for creation, last modification and last use.

The information about all files is kept in the directory structure which resides on secondary storage. A directory entry consists of the file's name and its unique identifier. The identifier in turn locates the other file attributes.

File Operations

A file is an abstract data type. OS can provide system calls to create, write, read, reposition, delete and truncate files.



Creating a file – First space in the file system must be found for the file. Second, an entry for the new file must be made in the directory.



Writing a file – To write a file, specify both the name of the file and the information to be written to the file. The system must keep a write pointer to the location in the file where the next write is to take place.



Reading a file – To read from a file, directory is searched for the associated entry and the system needs to keep a read pointer to the location in the file where the next read is to take place. Because a process is either reading from or writing to a file, the current operation location can be kept as a per process current file position pointer.



Repositioning within a file – Directory is searched for the appropriate entry and the current file position pointer is repositioned to a given value. This operation is also known as file seek.



Deleting a file – To delete a file, search the directory for the named file. When found, release all file space and erase the directory entry.



Truncating a file – User may want to erase the contents of a file but keep its attributes. This function allows all attributes to remain unchanged except for file length.

File types

A common technique for implementing file types is to include the type as part of the file name. The name is split into two parts – a name and an extension separated by a period character. The system uses the extension to indicate the type of the file and the type of operations that can be done on that file.

<u>FILE TYPE</u>	<u>EXTENSION</u>
Executable	exe, com, bin
Object	obj, o
Source code	c, cc, java
Batch	bat
Text	txt, doc
Word processor	rex, rtf
Library	lib
Multimedia	mp3, mpg, mpeg

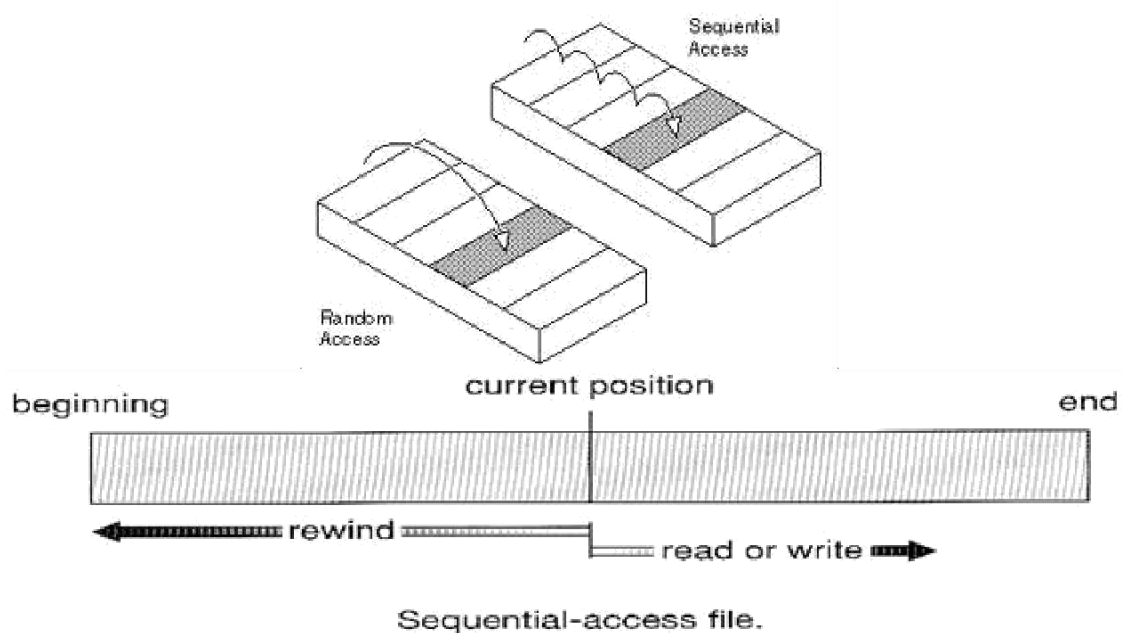
ACCESS METHODS:

Files store information. When it is used, this information must be accessed and read into computer memory. The information in the file can be accessed in several ways.

They are –

- * **Sequential access:** Simplest method. Information in the file is processed in order that is one record after the other. This method is based on a tape model of a file and works as well on sequential access devices as it does on random access .

A Read operation reads the next portion of the file and automatically advances a file pointer, which tracks the I/O location. This is done by using the **read next** operation. Similarly, the Write operation appends to the end of the file and advances to the end of the newly written material, this is done by using **write next** operation.



- * **Direct access:** Another method is direct access or relative access. A file is made up of fixed length logical records that allow programs to read and write records rapidly in no particular order. The direct access method is based on a disk model of a file since disks allow random access to any file block. Thus, we may read block 14 , then read block 53 and then write block 7. There are no restrictions on the order of reading or writing for a direct access file.
- * Direct access files are of great use for immediate access to large amounts of information. In this method, file operations must be modified to include block number as a parameter.

- * **Example:** in the airline reservation system, we might store all the information about a particular flight (eg: flight no. 713) in one block. Thus, the number of available seats for the flight 713 is stored in block 713 of the reservation file.

To perform the read operation we use **read n** operation , where **n** is the block number. To perform write operation we use **write n** operation, where **n** is the block number.

Some systems allow only sequential file access; others allow only direct access.

- **Other Access Methods:** Other access methods can be built on top of a direct access method. These methods generally involve the construction of an index for the file. This index contains pointers to the various blocks. To find a record in the file, first search the index and then use the pointer to access the file directly and to find the desired record.

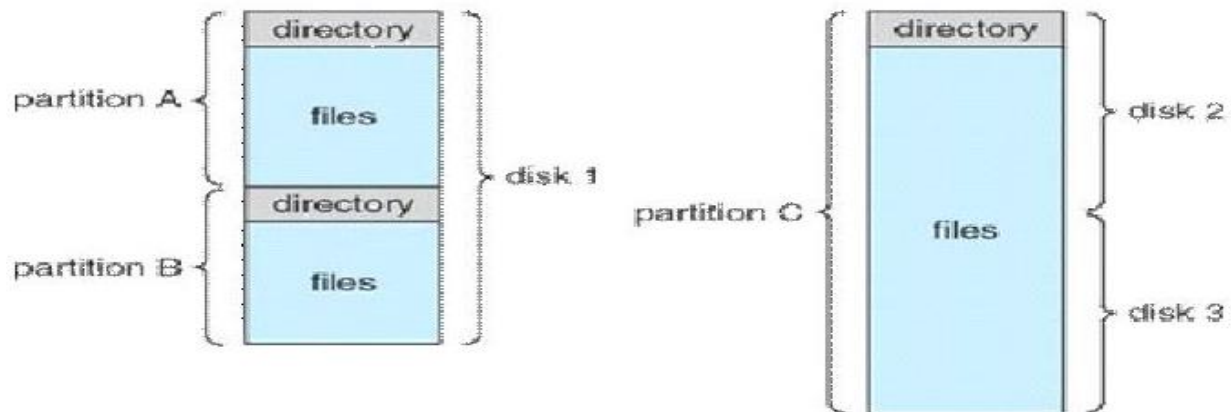
DIRECTORY STRUCTURE:

Systems may have zero or more file systems and the file systems may be of varying types. Organizing millions of files involves use of directories.

Storage Structure

A disk can be used in its entirety for a file system. But at times, it is desirable to place multiple file systems on a disk or to use parts of a disk for a file system and other parts for other things. These parts are known variously as **partitions, slices or minidisks**. A file system can be created on each of these parts of the disk. These parts can be combined together to form larger structures known as **volumes** and file systems can be created on these too. Each volume can be thought of as a virtual disk. Volumes can also store multiple OS's allowing a system to boot and run more than one.

Each volume that contains a file system must also contain information about the files in the system. This information is kept in entries in a **device directory or volume table of contents**. The device directory/directory records information for all files on that volume.



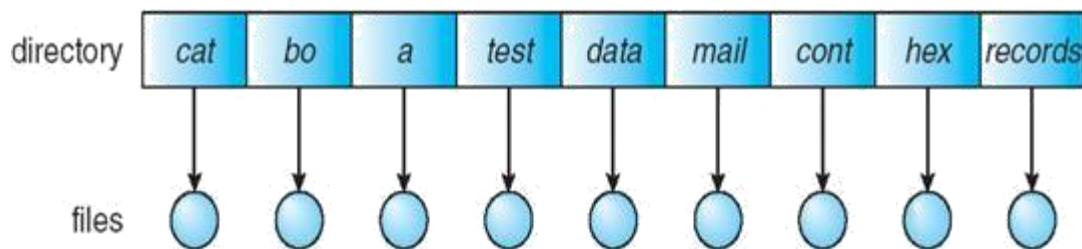
Directory Overview

The directory can be viewed as a symbol table that translates file names into their directory entries. The operations that can be performed on the directory are:

- ☒ **Search for a file** : used to search a directory structure to find the entry for a particular file.
- ☒ **Create a file** : new files can be created and added to the directory.
- ☒ **Delete a file** : when a file is no longer needed, we can delete it from the directory.
- ☒ **List a directory** : used to list the files in a directory.
- ☒ **Rename a file** : we can rename a file
- ☒ **Traverse the file system** : we may wish to access every directory and every file within a directory structure.

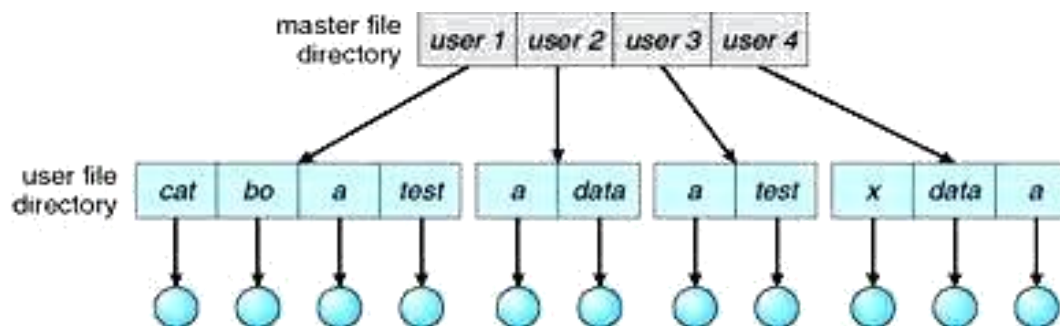
Single level directory

The simplest directory structure is the single level directory. All files are contained in the same directory which is easy to support and understand. But this implementation has limitations when the number of files increases or when the system has more than one user. Since all files are in same directory, **all files names must be unique**. Keeping track of so many files is a difficult task. A single user on a single level directory may find it difficult to remember the names of all the files as the number of files increases.



Two level directory

In the two level directory structure, each user has his own **user file directory** (UFD). The UFD's have similar structures but each lists only the files of a single user. When a user job starts or a user logs in, the system's **master file directory** (MFD) is searched. The MFD is indexed by user name or account number and each entry points to the UFD for that user. When a user refers to a particular file, only his own UFD is searched. Different users may have files with the same name as long as all the files names within each UFD are unique.



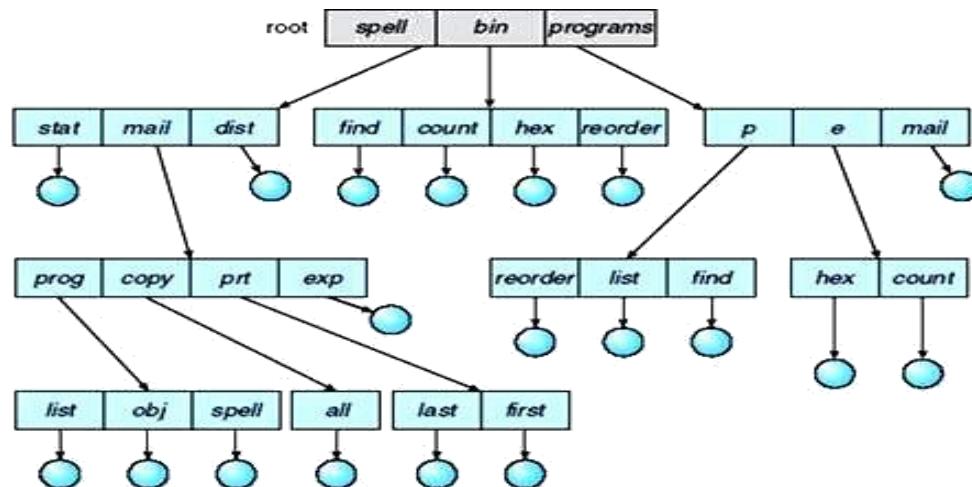
Root of the tree is MFD. Its direct descendants are UFDs. The descendants of the UFDs are the files themselves. The files are the leaves of the tree.

The sequence of directories searched when a file is names is called the **search path**.

Although the two level directory structure **solves the name collision problem**, it still has disadvantages. This structure **isolates on user from another**. Isolation is an advantage when the users are completely independent but a disadvantage when the users want to cooperate on some task and to access one another's files.

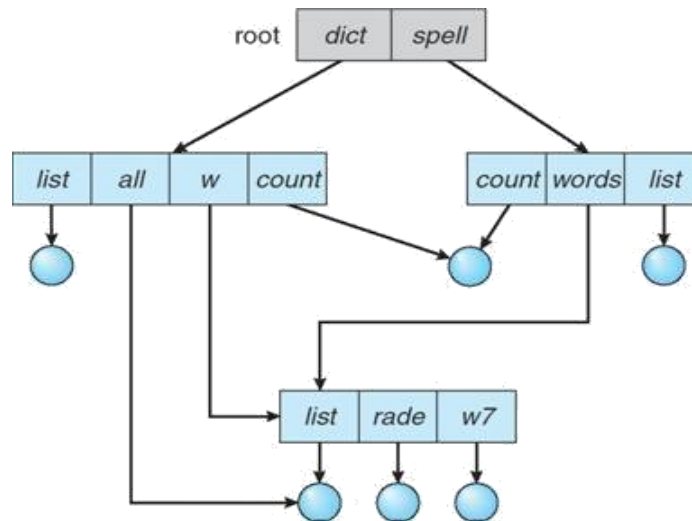
Tree Structured Directories

Here, we extend the two level directory to a tree of arbitrary height. This generalization allows users to create their own subdirectories and to organize their files accordingly. A tree is the most common directory structure. The tree has a root directory and every file in the system has a unique path name. A directory contains a set of files or sub directories. All directories have the same internal format. One bit in each directory entry defines the entry as a file (0) or as a subdirectory (1).



Shared file is not the same as two copies of the file. With two copies, each programmer can view the copy rather than original, but if one programmer changes the file, the changes will not appear in the other's copy. With a shared file, only one actual file exists, so any changes made by one person are immediately visible to the other.

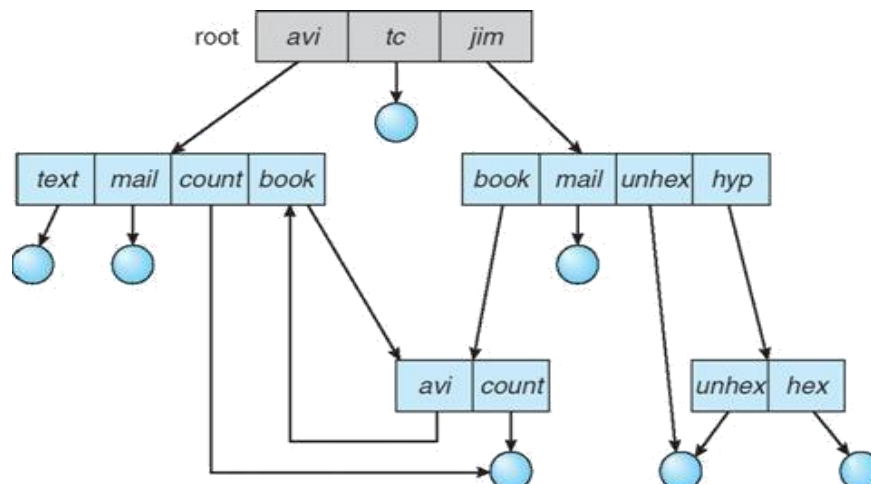
With a shared file, only one actual file exists. Sharing is particularly important for subdirectories. Shared files and subdirectories can be implemented in several ways. One way is to create a new directory entry called a link. A link is a pointer to another file or subdirectory. Another approach in implementing shared files is to duplicate all information about them in both sharing directories.



An acyclic graph directory structure is flexible than a tree structure but it is more complex. Several problems may exist such as multiple absolute path names or deletion.

General graph directory

A problem with using an acyclic graph structure is ensuring that there are no cycles.



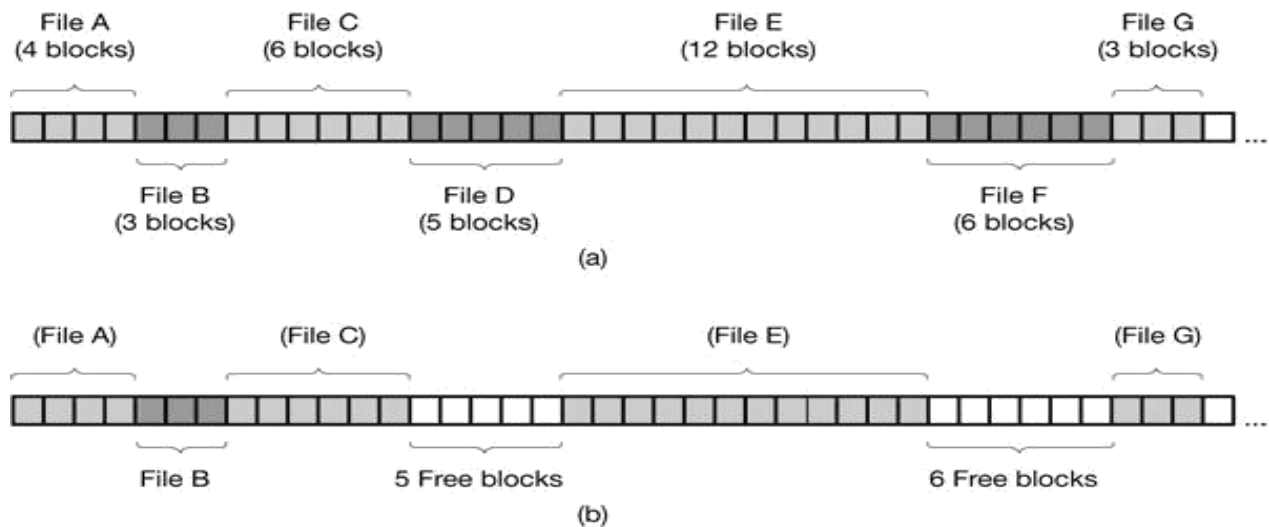
ALLOCATION METHODS:

The direct access nature of disks allows flexibility in the implementation of files. The main problem here is how to allocate space to these files so that disk space is utilized effectively and files can be accessed quickly. Three major methods of allocating disk space are:

- i. Contiguous
- ii. Linked
- iii. Indexed

Contiguous Allocation

This allocation requires that each file occupy a set of contiguous blocks on the disk. The number of disk seeks required for accessing contiguously allocated files is minimal. Contiguous allocation of a file is defined by the disk address and length of the first block. Accessing a file that has been contiguously allocated is easy. Both sequential and direct access can be supported by contiguous allocation. Disadvantage is finding space for a new file.



This problem can be seen as a particular application of general dynamic storage allocation problem which involves how to satisfy a request of size n from a list of free holes. First fit and best fit are the most common strategies used to select a free hole from the set of available holes.

These algorithms suffer from external fragmentation. As files are allocated and deleted, the free disk space is broken into little pieces. External fragmentation exists whenever free space is broken into chunks. For solving the fragmentation problem, compact all free space into one contiguous space.

Another problem with contiguous allocation is determining how much space is needed for a file.

Pre allocation of memory space to a file may be insufficient. A file may be allocated space for its

final size but large amount of that space will remain unused for a long time. The file therefore has a large amount of internal fragmentation.

To minimize these drawbacks, some operating systems use a modified contiguous allocation scheme. Here a contiguous chunk of space is allocated initially and if that amount proves not to be large enough another chunk of contiguous space called **extent** is added. Internal fragmentation can still be a problem if the extents are too large and external fragmentation can become a problem as extents of varying sizes are allocated and de allocated.

Linked Allocation

This solves all problems of contiguous allocation. Each file is a linked list of disk blocks; the disk blocks may be scattered anywhere on the disk. The directory contains a pointer to the first and last blocks of the file. Each block contains a pointer to the next block. There is no external fragmentation with linked allocation and any free block on the free space list can be used to satisfy a request. But the major problem is that it can be used effectively only for sequential access files. It is inefficient to support a direct access capability for linked allocation files. Another disadvantage is space required for pointers.

Solution to this problem is to collect blocks into multiples called clusters and to allocate clusters rather than blocks. This method allows logical to physical block mapping to remain simple but improved disk throughput and decreases the space needed for block allocation and free list management. This increases internal fragmentation because more space is wasted when a cluster is partially full than when a block is partially full. Another problem of linked allocation is reliability.

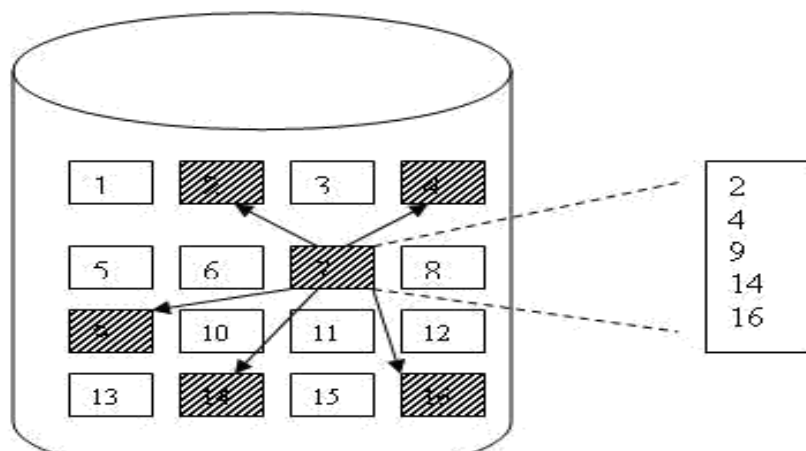
An important variation of linked allocation is the use of a file allocation table (FAT).

Refer Fig No: 11.6 Linked allocation of disk space

Indexed Allocation

Linked allocation solves external fragmentation and size declaration problems of contiguous allocation. In the absence of FAT, linked allocation cannot support efficient direct access since the pointers to the blocks are scattered with the blocks themselves all over the disk and must be retrieved in order. Indexed allocation solves this problem by bringing all pointers together into one location – index block.

Each file has its own index block which is an array of disk block addresses.



Indexed allocation supports direct access without suffering from external fragmentation because any free block on the disk can satisfy a request for more space. But indexed allocation suffers from wasted space. Every file must have an index block so it should be as small as possible. But if it is too small, it will not be able to hold enough pointers for a large file and a mechanism will have to be available to deal with this issue. Mechanisms for this purpose include –

1. Linked scheme
2. Multilevel index
3. Combined scheme

Indexed allocation scheme suffers from some of the same performance problems as does linked allocation.

Performance

The allocation methods vary in their storage efficiency and data block access times. Both are important in selecting the proper method for an operating system to implement. Before selecting an allocation method, determine how systems will be used.

For any type of access, contiguous allocation requires only one access to get a disk block. For linked allocation, we can keep the address of the next block in memory and read it directly. This method is fine for sequential access. Hence some systems support direct access files by using contiguous allocation and sequential access by linked allocation.

UNIT – V

Protection

Protection refers to mechanisms for controlled access of programs, processes, or users to the resources. So protection mechanisms ensure that, users or processes can have controlled accesses to the resources such as memory segments, CPU, files, etc. The authorized processes or users only can have access to resources.

GOALS OF PROTECTION

The system should prevent misuse of system resources by users or processes. It should also ensure that each shared resource is used by users or processes according to the system policies set by the system administrator. We need to provide protection for several reasons- one is the need to prevent mischievous, intentional violation of an access restriction by a user. The role of protection in a computer system is to provide a mechanism for the enforcement of the policies governing resource use. Protection problem - ensure that each object is accessed correctly and only by those processes that are allowed to do so. The protection to the system resources will be provided by implementing protection mechanisms for the policies set by the administrator.

PRINCIPLE OF PROTECTION

The programs, and users of the system must be given enough privileges (rights or authorization) to perform their tasks. This ensures that tasks will be completed by the users without damaging or violating the protection to the resources. Typically each user is given their own account, and has only enough privilege to modify their own files.

DOMAIN OF PROTECTION

A computer system is a collection of processes and objects. The objects may be **hardware objects** (such as CPU, memory segments, printers, disks, tape drives, etc), and **software objects** (such as files, programs, and semaphores, etc). Each object has unique name and can be accessed through well-defined operation. For example, memory segments can be read and written, data files can be created, opened, read, written, close and delete and program files can be read, write, execute and delete. A process is allowed to access the resources for which it has authorization. The ***need to know principle*** states that a process should only have access to those objects it needs to accomplish its task.

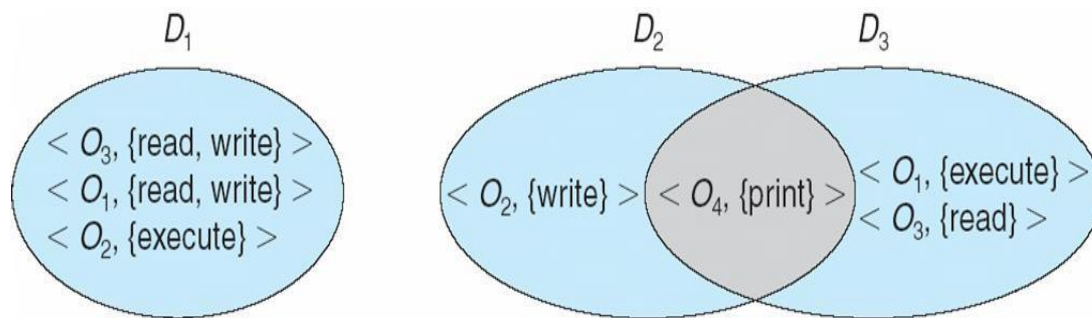
Domain structure

A process operates within a protection domain, which specifies the resources or objects that the process may access. Each domain defines a set of objects and the types of operations that may be invoked on each object.

An ***access right*** is the ability to execute an operation on an object. A domain is defined as a set of < object, { access right set } > pairs. For example, if domain D has access right <file F, {read, write}>,

then the process executing in domain can both have read and write F. Some domains may be disjoint while others may overlap.

System with three protection domains



For example, three domains D1, D2, and D3 exist. The access right $\langle O_4, \{print\} \rangle$ is shared by both D2 and D3, and it indicates that a process executing in either of these domains can print object O4.

A process executes in domain D1 to read and write object O1 and only one process in domain D3 may execute object O1.

The association between a process and a domain may be either **static**, if the set of resources available to the process is fixed throughout the process's lifetime, or **dynamic**. If the association is dynamic, a mechanism is available to allow domain switching, enabling the process to switch from one domain to another.

A domain can be realized in a variety of ways:

- Each *user* may be a domain. Domain switching occurs when the user is changed—generally when one user logs out and another user logs in.
- Each *process* may be a domain. Domain switching occurs when one process sends a message to another process and then waits for a response.
- Each *procedure* may be a domain. Domain switching occurs when a procedure call is made.

Domain Example1 (UNIX):

Domain switch accomplished via file system.

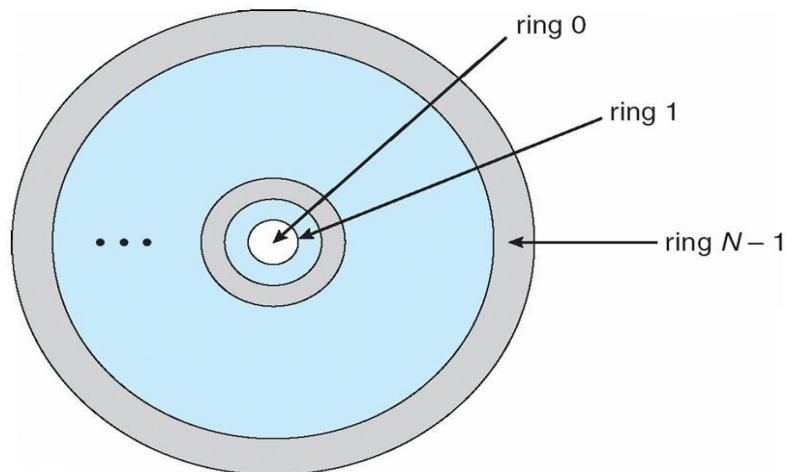
- ▶ Each file has associated with it a domain bit (setuid bit)
- ▶ When file is executed and setuid = on, then user-id is set to owner of the file being executed, when the bit is *off* however, the user ID does not change.
 - For example, when a user A (that is, a user with *userID* = A) starts executing a file owned by B, whose associated domain bit is *off*, the

userID of the process is set to *A*. When the *setuid* bit is *on*, the *userID* is set to that of the owner of the file: *B*.

- ▶ When execution completes user-id is reset .

Domain Example2 (MULTICS):

- In the MULTICS system, the protection domains are organized hierarchically into a ring structure. Each ring corresponds to a single domain .
- The rings are numbered from 0 to 7.
- Let D_i and D_j be any two domain rings. If $j < i$, then D_i is a subset of D_j . That is, a process executing in domain D_j has more privileges than does a process executing in domain D_i .
- A process executing in domain D_0 has the most privileges.



A *current-ring-number* counter is associated with each process, identifying the ring in which the process is executing currently. When a process is executing in ring i , it cannot access a segment associated, with ring j ($j < i$). It can access a segment associated with ring k ($k > i$).

ACCESS MATRIX

The model of protection can be viewed as a matrix, called an **access matrix**. The rows of access matrix represents domains, and the columns represent objects. Each entry in the access matrix consists of a set of access rights. The entry $access(i, j)$ defines the set of operations that a process executing in domain D_i can invoke on object O_j .

Consider four domains and four objects, three files (F1, F2, F3) and one printer. The process executing in domain D_1 , can read files F1 and F3. A process execute in domain D_4 , can read and

write files F_1 and F_3 . A process execute in domain D_2 can access the printer.

object domain	F_1	F_2	F_3	printer
D_1	read		read	
D_2				print
D_3		read	execute	
D_4	read write		read write	

The association between a process and a domain may be *static* or *dynamic*. If the process is in the same domain, then association is said to be static, and if a process switches (moves) from one domain to another domain, then association is said to be dynamic. When a process is switching from one domain to another domain, we are executing an operation (switch) on an object (the domain). The processes can switch from one domain to another domain. The domain switching from one domain to another domain is allowed if $\text{access}(i,j)=\text{switch}$.

The example access matrix with domains as objects is shown below. Process executing in domain D_1 , can switch to domain D_2 . A process executing in domain D_2 can switch to domain D_3 or domain D_4 .

object domain	F_1	F_2	F_3	laser printer	D_1	D_2	D_3	D_4
D_1	read		read			switch		
D_2				print			switch	switch
D_3		read	execute					
D_4	read write		read write		switch			

The contents of access matrix entries can be changed by using three additional operations: copy, owner, and control. The ability to copy an access right from one domain (or row) of the access matrix to another domain is denoted by an asterisk (*) appended to the access right. The **copy** right allows the copying of the access right only within the column (i.e., for the object) for which the access right is defined.

This scheme has two variants:

1. A right access is copied from $\text{access}(i,j)$ to $\text{access}(k,j)$. Then it is removed from $\text{access}(i,j)$. This action is **transfer** of right, rather than copy.
2. When the right R^* is copied from $\text{access}(i,j)$ to $\text{access}(k,j)$, only the right R (not R^*) is created. A process executing in Domain D_k cannot further copy the right R . This is known as **limited copy**.

The access matrix with copy rights is shown below:

object domain	F_1	F_2	F_3
D_1	execute		write*
D_2	execute	read*	execute
D_3	execute		

(a)

object domain	F_1	F_2	F_3
D_1	execute		write*
D_2	execute	read*	execute
D_3	execute	read	

(b)

The **owner** right control allows addition of new rights or removal of some rights on an object. If access(i,j) is the **owner** right, then process executing in domain D_i can add or remove any entry in column j. For example, domain D_1 is the owner of file F_1 , so it can add or delete any valid right in column F_1 . Similarly, domain D_2 is the owner of F_2 and F_3 , so it can add or remove any valid right within these two columns.

object domain	F_1	F_2	F_3
D_1	owner execute		write
D_2		read* owner	read* owner write
D_3	execute		

(a)

object domain	F_1	F_2	F_3
D_1	owner execute		write
D_2		owner read* write*	read* owner write
D_3		write	write

(b)

The **copy** and **owner** rights allows a process to change the entries in a column. The **control** right allows a process to change the entries in row. If access(i,j) includes **control** right, then a process executing in domain D_i can change any access right in row j. The access(D_2, D_4) is control. Then the process executing in domain D_2 could modify the entries for domain D_4 .

object domain	F_1	F_2	F_3	laser printer	D_1	D_2	D_3	D_4
D_1	read		read			switch		
D_2				print			switch	switch control
D_3		read	execute					
D_4	write		write		switch			

Security

The information stored in the system (both code and data), as well as physical resources of the computer system, need to be protected from unauthorized access, and malicious destruction or alteration.

THE SECURITY PROBLEM

The system is said to be secure if its resources are used and accessed as intended under all circumstances. Security violations (or misuse) of the system can be categorized as intentional (malicious) or accidental. It is easier to protect against accidental misuse.

The several forms of accidental and malicious security violations are:

- (i) Breach of confidentiality: This type of violation involves unauthorized reading of data (or theft of information).
- (ii) Breach of integrity: This violation involves unauthorized modification of data.
- (iii) Breach of availability: This violation involves unauthorized destruction of data.
- (iv) Theft of service: This violation involves unauthorized use of resources.
- (v) Denial of service: This violation involves preventing legitimate use of the system.

Security Violation Methods:

1. **Masquerading (breach authentication)**
 - Pretending to be an authorized user to escalate privileges
2. **Replay attack**
 - consists of the malicious or fraudulent repeat of a valid data transmission.
3. **Man-in-the-middle attack**
 - Intruder sits in data flow, masquerading as sender to receiver and vice versa
4. **Session hijacking**
 - in which an active communication session is intercepted

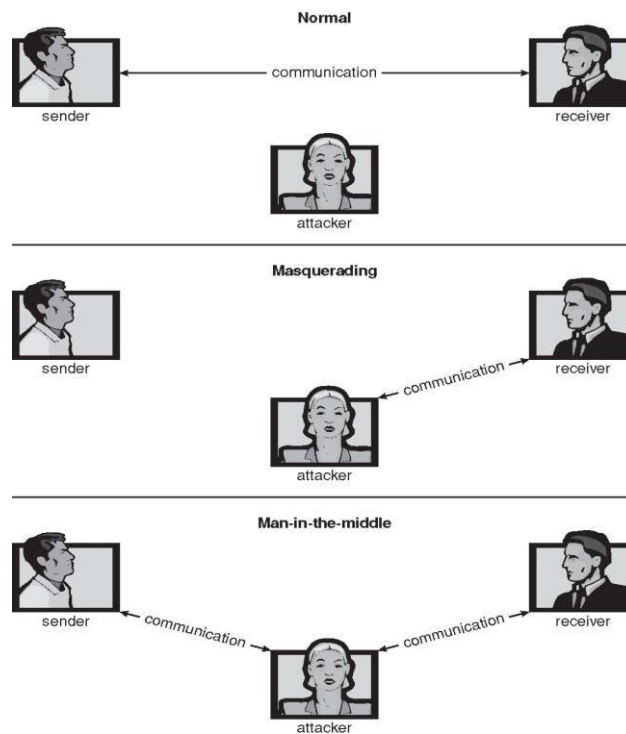
There are four levels of security measures to protect the system :

- (i) Physical: The site or sites containing the computer systems must be physically secured against entry by intruders.
- (ii) Human: Users must be screened (monitored and checked) to reduce the chance of authorizing a user who then gives access to an intruder.

(iii) Network: Much computer data may be transmitted in network and security to be given at network level.

(iv) operating system: The operating system services must protect itself from accidental or malicious attacks.

Standard Security Attacks:



To protect a system, we must take security measures at four levels:

1. Physical
2. Human
3. Operating System
4. Network

PROGRAM THREATS

The operating system's processes and kernel carry out the specified task as directed. Program Threats occur when a user program causes these processes to do malicious operations. The common example of a program threat is that when a program is installed on a computer, it could store and transfer user credentials to a hacker. There are various program threats. Some of them are as follows:

1. Virus

A virus may replicate itself on the system. Viruses are extremely dangerous and can modify/delete user files as well as crash computers. A virus is a little piece of code

that is implemented on the system program. As the user interacts with the program, the virus becomes embedded in other files and programs, potentially rendering the system inoperable.

2. Trojan Horse

This type of application captures user login credentials. It stores them to transfer them to a malicious user who can then log in to the computer and access system resources.

3. Logic Bomb

A logic bomb is a situation in which software only misbehaves when particular criteria are met; otherwise, it functions normally.

4. Trap Door

A trap door is when a program that is supposed to work as expected has a security weakness in its code that allows it to do illegal actions without the user's knowledge.

5. Stack and Buffer Overflow

- Exploits a bug in a program (overflow either the stack or memory buffers)
- Failure to check bounds on inputs, arguments
- Write past arguments on the stack into the return address on stack
- When routine returns from call, returns to hacked address
 - Pointed to code loaded onto stack that executes malicious code
- Unauthorized user or privilege escalation

C Program with Buffer-overflow Condition:

```
#include <stdio.h>
#define BUFFER SIZE 256
int main(int argc, char *argv[])
{
    char buffer[BUFFER SIZE];
    if (argc < 2)
        return -1;
    else {
        strcpy(buffer,argv[1]);
        return 0;
    }
}
```

System Threats

System threats are described as the misuse of system services and network connections to cause user problems. These threats may be used to trigger the program threats over an entire network, known as program attacks. System threats make an environment in which OS resources and user files may be misused. There are various system threats. Some of them are as follows:

1. Port Scanning

It is a method by which the cracker determines the system's vulnerabilities for an attack. It is a fully automated process that includes connecting to a specific port via TCP/IP. To protect the attacker's identity, port scanning attacks are launched through Zombie Systems, which previously independent systems now serve their owners while being utilized for such terrible purposes.

2. Worm

The worm is a process that can choke a system's performance by exhausting all system resources. A Worm process makes several clones, each consuming system resources and preventing all other processes from getting essential resources. Worm processes can even bring a network to a halt.

USER AUTHENTICATION

Authentication is based on one or more three items: user possession (a key or card), user knowledge (a user identifier and password), and a user attribute (fingerprint, retina pattern or signature).

Passwords:

The most common approach to authenticating a user identity is the use of passwords. Passwords are used to protect the objects in the computer system. A password may be associated with each resource (such as a file). When the user wants to use the resource, the user has to enter the password. If the password is correct, access is granted.

Password Vulnerabilities:

passwords can be often guessed, accidentally exposed, sniffed, or illegally transferred from an authorized user to an unauthorized user. There are two ways to guess a password. One way is guess password from information about the user, i.e. user's name, interests, etc. other way is brute force, i.e., trying with the combination of letters, numbers, special characters, special symbols until the password is found.

The password is accidentally exposed from visual monitoring of password by the intruder, when the user is logging in. The user having access in the network, can monitor all the data containing password transferring on the network.

Passwords can be generated by the system or selected by a user. The system generated passwords may be difficult to remember, and thus users may write them down. User selected passwords are easy to guess. Some systems may force the users to change their passwords at regular intervals. These systems do not allow the user to select a password that was used earlier.

Encrypted passwords

The password can be stored securely by encrypting it to another form. Each user has a password. The system uses encryption function for converting a password into encrypted form. The passwords in encrypted format will be stored in the system. If intruder gets the encrypted password from the system, he can not decode it to get the original password.

One time passwords

In one time password system, the password is different in each instance or session. Any one capturing password from and trying to use it in another session will fail. The system generates one time password (OTP) and it will be sent to the user. The user has to enter PIN (Personal Information Number) or password along with one time password for authentication purpose. This authentication process is also known as Two-factor authentication. When a session begins, the system presents one part of a password pair (i.e. OTP) to the user and the user supplies the other part of password pair (i.e. PIN or password).

Biometrics

The system uses Biometric mechanism for secure access to the system resources. The system uses finger print reader device to scan the user's finger image and the scanned image will be stored. When user wants to access the system resources, the device scans the user's finger image and compares it to the stored images . If the match is found, then only the user is allowed to access the system resources.